

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2025/2026

DECEMBER EXAMINATION

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

SATURDAY, 27 DECEMBER 2025

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Instructions to Candidates:

This question paper consists of FIVE (5) questions.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**. Each question carries 25 marks.

Should a candidate answers more than ONE (1) question in section B, marks will only be awarded for the FIRST question in that section in the order the candidate submits the answers.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION A (Answer all questions)**

Q1. (a) Gradient Descent is an optimization algorithm that minimizes a function.

- (i) Explain the main idea behind Gradient Descent. (3 marks)
- (ii) What will happen if the *learning rate* is set too high or low? (4 marks)
- (iii) Explain how the computed *gradient* helps to find the minimum of a function. (3 marks)

(b) Gradient Descent is used to minimize the following function:

$$f(x) = 4x^2 - 8x + 6$$

- (i) Express the derivative of $f(x)$. (3 marks)
- (ii) Assuming the learning rate is correctly set, will Gradient Descent always reach the global minimum for this function? Explain your answer. (3 marks)
- (iii) Suppose the initial value of $x = 2$, and the learning rate is $\alpha = 0.2$. Carry out Gradient Descent for **THREE (3)** iterations and record the values of x , $f(x)$ and the gradient $\frac{df}{dx}$ at each step. Round all results to two decimal places.

Iteration (t)	x	$f(x)$	df/dx
1			
2			
3			

(9 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Q2. (a) Answer the following questions on Convolutional Neural Network (CNN).

- (i) What is the role of the following layers in a CNN?
 - Convolutional Layer
 - Pooling Layer
 - Fully Connected Layer

(6 marks)
- (ii) State **TWO (2)** advantages of CNNs over fully connected networks for image data.

(4 marks)
- (iii) Compare a filter (in a convolutional layer) and a neuron (in a fully connected layer).

(6 marks)
- (iv) A convolutional layer is defined by the number of filters k , filter size f , stride s , padding p . Identify the hyperparameter that affects the number of output channels of the layer. Explain your answer.

(3 marks)

(b) A convolutional layer receives the following 4×4 input activation map:

$$\begin{bmatrix} 1 & 0 & 2 & 3 \\ 4 & 6 & 6 & 8 \\ 3 & 1 & 1 & 0 \\ 1 & 2 & 2 & 4 \end{bmatrix}$$

and applies a 3×3 filter (stride = 1, padding = 0) with weights:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Compute the resulting output feature map after applying the convolution operation.

(6 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

- Q3. (a) (i) Explain why a Recurrent Neural Network (RNN) is more suitable at handling sequence data than a Standard Neural Network (or equivalently Multilayer Perceptron). (3 marks)
- (ii) Describe the role of the hidden state in an RNN and how it helps to process sequential data. (3 marks)
- (iii) State and explain **ONE (1)** common problem that occurs when training RNNs on long sequence. (3 marks)
- (b) You are designing an RNN to predict if each word in the sentence is a named entity. The model uses:
- An input sequence length of 20 words
 - Each word is represented by an embedding vector of length 100
 - RNN configuration (10 hidden units, 1 layer, `bidirectional=True`)
 - The batch size during training is 16
- (i) Which input-output configuration (many-to-one, many-to-many, one-to-one or encoder-decoder) is appropriate for this task? Justify your answer. (3 marks)
- (ii) Draw the unrolled network for this problem. (6 marks)
- (iii) State the *input* and *output* shape of the RNN for the given configuration. (4 marks)
- (iv) Explain why enabling the `bidirectional` setting enhances the model's performance. (3 marks)
- [Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION B (Choose ONE question only)**

Q4. (a) Figure 4.1 shows a Transformer Network.

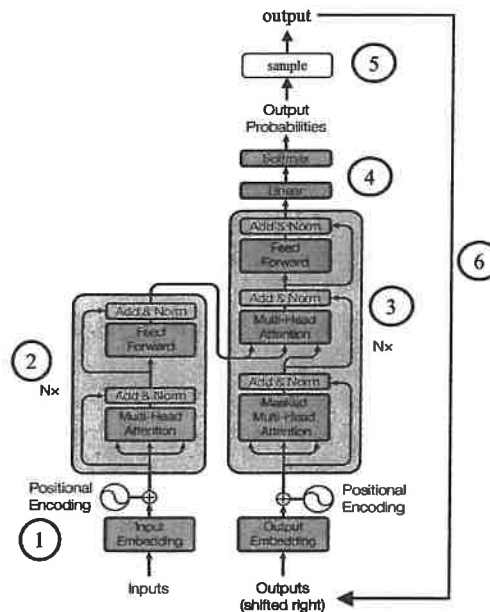


Figure 4.1 Transformer Network

Suppose that the transformer is used to perform machine translation to translate a Chinese text to English text. Explain how each of components in Figure 4.1 processes the Chinese text to output the equivalent English text.

- | | | |
|-------|-----------------|-----------|
| (i) | Input block ① | (3 marks) |
| (ii) | Encoder Block ② | (3 marks) |
| (iii) | Decoder Block ③ | (3 marks) |
| (iv) | Output Block ④ | (3 marks) |
| (v) | Sampling ⑤ | (3 marks) |
| (vi) | Feedback path ⑥ | (3 marks) |
- (b)
- Explain the main purpose of the *attention* mechanism in a Transformer model. (2 marks)
 - Describe the difference between *self*-attention and *cross*-attention in the Transformer architecture. (3 marks)
 - State **ONE** (1) advantage of using *multi-head attention* over single-head attention in Transformers. (2 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

- Q5. (a) Temperature Decoding can be used to control the output of a Large language Model (LLM).
- (i) Suppose $T > 0$. Explain how a *low* and *high* setting of the temperature T influence the randomness of the generated text, respectively. (5 marks)
 - (ii) What would be the effect of setting the temperature value to 0? (3 marks)
 - (iii) Suppose $T = 0.7$. The language model produces repetitive text when generating summaries. Suggest an appropriate adjustment to the temperature value and justify your choice. (4 marks)
- (b) Answer the following questions on Beam search.
- (i) Describe how *beam search* differs from *greedy decoding* in generating text sequences. (5 marks)
 - (ii) Define the term *beam width* and explain its role in controlling the *diversity* and *computational* of generated sequences. (5 marks)
 - (iii) A real-time chatbot needs to generate responses instantly for user queries. Why is the beam search decoding **NOT** appropriate for this task? (3 marks)
- [Total : 25 marks]
-

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Logistic Regression

Forward propagation

$$z = w_1 x_1 + \dots + w_{F_x} x_{F_x} + b = \mathbf{x}^T \mathbf{w} + b$$

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}}$$

Gradient Descent

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{1}{M} \left(\frac{1-y}{(1-\hat{y})} - \frac{y}{\hat{y}} \right)$$

$$\frac{\partial J}{\partial \mathbf{z}} = \frac{\partial J}{\partial \hat{y}} * (\hat{\mathbf{y}} * (1 - \hat{\mathbf{y}}))$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{z}}$$

$$\frac{\partial J}{\partial b} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}} \right)^T$$

Standard Neural Network

Forward propagation

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$$

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \text{ (softmax)}$$

Gradient Descent

$$\frac{\partial J}{\partial \hat{\mathbf{y}}} = \frac{1}{C \times B} \left(\frac{1-y}{1-\hat{y}} - \frac{y}{\hat{y}} \right)$$

$$\frac{\partial J}{\partial \mathbf{Z}^{[l]}} = \begin{cases} \frac{\partial J}{\partial \mathbf{A}^{[l]}} * \mathbf{A}^{[l]} * (1 - \mathbf{A}^{[l]}) & \text{for sigmoid activation} \\ \frac{\partial J}{\partial \mathbf{A}^{[l]}} * (\mathbf{Z}^{[l]} > 0) & \text{for relu activation} \end{cases}$$

$$\frac{\partial J}{\partial \mathbf{w}^{[l]}} = \mathbf{A}^{[l-1]T} \frac{\partial J}{\partial \mathbf{Z}^{[l]}}$$

$$\frac{\partial J}{\partial \mathbf{b}^{[l]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \right)^T$$

$$\frac{\partial J}{\partial \mathbf{A}^{[l-1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \mathbf{W}^{[l]T}$$

Activation Functions

Sigmoid

$$g(z) = \frac{1}{1+e^{-z}}, \quad \frac{dg}{dz} = g(z) \cdot (1 - g(z))$$

Tanh

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \frac{dg}{dz} = 1 - g(z)^2$$

ReLU

$$g(z) = \max(0, z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Leaky ReLU

$$g(z) = \max(0.01z, z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0.01 & \text{if } z < 0 \end{cases}$$

RNN and LSTM

RNN

$$\mathbf{H}^{<t>} = g(\mathbf{H}^{<t-1>} \mathbf{W}_{hh} + \mathbf{X}^{<t>} \mathbf{W}_{hx} + \mathbf{b}^T)$$

$$\mathbf{H}^{<t>} = g([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_h + \mathbf{b}^T)$$

LSTM

$$\mathbf{F}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_f + \mathbf{b}_f^T)$$

$$\mathbf{I}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_i + \mathbf{b}_i^T)$$

$$\mathbf{O}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_o + \mathbf{b}_o^T)$$

$$\tilde{\mathbf{C}}^{<t>} = \tanh([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_c + \mathbf{b}_c^T)$$

$$\mathbf{C}^{<t>} = \mathbf{I}^{<t>} * \tilde{\mathbf{C}}^{<t>} + \mathbf{F}^{<t>} * \mathbf{C}^{<t-1>}$$

$$\mathbf{H}^{<t>} = \mathbf{O}^{<t>} * \tanh(\mathbf{C}^{<t>})$$

Cost Function

Binary cross entropy

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln (1 - \hat{y}^{(i)})$$

$$= \text{mean}(-\mathbf{y} * \ln \hat{\mathbf{y}} - (1 - \mathbf{y}) * \ln(1 - \hat{\mathbf{y}}))$$

Binary cross entropy (multi-label)

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{C \times B} \sum_{c=1}^C \sum_{i=1}^B -y_c^{(i)} \ln \hat{y}_c^{(i)} - (1 - y_c^{(i)}) \ln (1 - \hat{y}_c^{(i)})$$

$$= \text{mean}(-\mathbf{Y} * \ln \hat{\mathbf{Y}} - (1 - \mathbf{Y}) * \ln(1 - \hat{\mathbf{Y}}))$$

Negative Log Likelihood

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -\ln(p^{(i)})$$

Convolutional Neural Network

$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

$$o = \left\lfloor \frac{i+2p-f}{s} + 1 \right\rfloor$$

$$\# \text{params in a filter} = c_f \times h_f \times w_f + 1$$

$$\# \text{params in a CONV layer} = \# \text{filters} \times \# \text{params in a filter}$$

$$\# \text{computations} = \# \text{output activations} \times \# \text{computations per activation}$$

$$= \text{output volume size} \times \# \text{params in a filter}$$

Efficient Net

$$\text{depth } d = \alpha^\phi, \text{ width } w = \beta^\phi, \text{ resolution } r = \gamma^\phi$$

$$\text{subject to: } \alpha, \beta^2, \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Transformers

$$\hat{\mathbf{V}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\hat{\mathbf{V}}_m = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{M}}{\sqrt{d_k}}\right) \mathbf{V}$$

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Optimizers

Momentum

$$\begin{aligned} \mathbf{v}^{<t>} &:= \beta \mathbf{v}^{<t-1>} + (1 - \beta) \frac{\partial J^{<t>}}{\partial \mathbf{w}} \\ \mathbf{v}^{<t>} &:= \frac{1}{1 - \beta^t} \mathbf{v}^{<t>} \\ \mathbf{w}^{<t>} &:= \mathbf{w}^{<t-1>} - \alpha \mathbf{v}^{<t>} \\ \text{weight}^{<t-n>} &= (1 - \beta)^n \end{aligned}$$

RMSProp

$$\begin{aligned} \mathbf{s}^{<t>} &:= \beta \mathbf{s}^{<t-1>} + (1 - \beta) \left(\frac{\partial J^{<t>}}{\partial \mathbf{w}} \right)^2 \\ \mathbf{w}^{<t>} &:= \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\mathbf{s}^{<t>} + \epsilon}} \right) \frac{\partial J^{<t>}}{\partial \mathbf{w}} \end{aligned}$$

Adam

$$\begin{aligned} \mathbf{v}^{<t>} &:= \beta_1 \mathbf{v}^{<t-1>} + (1 - \beta_1) \frac{\partial J^{<t>}}{\partial \mathbf{w}} \\ \hat{\mathbf{v}}^{<t>} &= \frac{\mathbf{v}^{<t>}}{(1 - \beta_1^t)} \\ \mathbf{s}^{<t>} &:= \beta_2 \mathbf{s}^{<t-1>} + (1 - \beta_2) \left(\frac{\partial J^{<t>}}{\partial \mathbf{w}} \right)^2 \\ \hat{\mathbf{s}}^{<t>} &= \frac{\mathbf{s}^{<t>}}{(1 - \beta_2^t)} \\ \mathbf{w}^{<t>} &= \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\hat{\mathbf{s}}^{<t>} + \epsilon}} \right) \hat{\mathbf{v}}^{<t>} \end{aligned}$$

Learning Rate Decay

$$\begin{aligned} \alpha^{<t>} &= \frac{1}{1 + rt} \alpha^{<0>} \\ \alpha^{<t>} &= \gamma^{t/T} \alpha^{<0>} \\ \alpha^{<t>} &= e^{-k \times t} \alpha^{<0>} \end{aligned}$$

Initialization

Xavier Initialization

$$\text{var}(\mathbf{W}^{[l]}) = \frac{2}{(F^{[l-1]} + F^{[l]})}$$

Kaiming Initialization

$$\text{var}(\mathbf{W}^{[l]}) = \frac{2}{F^{[l-1]}}$$

Normalization

BatchNorm

$$\begin{aligned} \hat{\mathbf{Z}}[:, f] &= \frac{\mathbf{Z}[:, f] - \mu_f}{\sqrt{v_f + \epsilon}} \\ \hat{\mathbf{Z}}[:, c, :, :] &= \frac{\mathbf{Z}[:, c, :, :] - \mu_c}{\sqrt{v_c + \epsilon}} \\ \hat{\mathbf{Z}}[:, c, :, :] &= \gamma_c \hat{\mathbf{Z}}[:, c, :, :] + \beta_c \\ \mu_c^{(t)} &= \text{mean}(\mathbf{Z}^{(t)}[:, c, :, :]) \quad (\text{training mode}) \\ v_c^{(t)} &= \text{variance}(\mathbf{Z}^{(t)}[:, c, :, :]) \quad (\text{training mode}) \\ \hat{\mu}_c &= (1 - m) \hat{\mu}_c + m \mu_c^{(t)} \quad (\text{eval mode}) \\ \hat{v}_c &= (1 - m) \hat{v}_c + m v_c^{(t)} \quad (\text{eval mode}) \end{aligned}$$

LayerNorm

$$\begin{aligned} \hat{\mathbf{Z}}[i, t, :] &= \frac{(\mathbf{Z}[i, t, :] - \mu^{(i)<t>})}{\sqrt{v^{(i)<t>} + \epsilon}} \\ \hat{\mathbf{Z}}[i, t, :] &= \gamma^{(i)<t>} \hat{\mathbf{Z}}[i, t, :] + \beta^{(i)<t>} \\ \mu^{(i)<t>} &= \text{mean}(\mathbf{Z}[i, t, :]) \quad (\text{training and eval mode}) \\ v^{(i)<t>} &= \text{variance}(\mathbf{Z}[i, t, :]) \quad (\text{training and eval mode}) \end{aligned}$$

Sampling Strategy

$$\begin{aligned} z'_i &= \frac{z_i}{\text{repeat}_{\text{count}} * \text{penalty_factor}} \quad (\text{repeat penalty}) \\ p'(w_i) &= \frac{\exp(\frac{z_i}{T})}{\sum_j \exp(\frac{z_j}{T})} \quad (\text{temperature decoding}) \end{aligned}$$

LLM Evaluation

$$\text{prec}(\hat{\mathbf{y}}) = \sum_i \frac{\text{exist_in_any_ref}(\hat{y}_i)}{|\hat{\mathbf{y}}|}$$

(Standard precision)

$$p_n = \frac{\sum_{w \in \text{ngram}} \min(\text{count}_{\text{LLM}}(w), \max(\text{count}_{\text{gt}}(w)))}{\sum_{w \in \text{ngram}} \text{count}_{\text{LLM}}(w)}$$

(Modified precision)

$$\text{bleu score} = \beta \exp(\sum_{n=1}^N w_n \ln p_n)$$

(Bleu score)